INF205 Summary Report – Group 14 – Erling Hamil Mysen & Kristian Mathias Røhne

2.1 – Graph data structure based on incidence lists

We implemented a linked graph data structure using incidence lists. The two main classes are Node and Edge. Each Node holds a label (unique string) and two lists of pointers to its incident edges – one for outgoing edges and one for incoming. Each Edge holds a label (non-unique), and pointers to its source and target nodes. The main Graph class stores all nodes and edges in separate linked lists and provides an *insert_edge()* method that creates new node objects automatically if they don't already exists.

2.2 – File input/output

The graph can be read from and written to a plain text file in triple notation, where each line has the format: *node_a_label edge_label node_b_label*. We parse this with a simple string stream and call insert_edge() for each line. Writing just iterates over all edges and outputs, in the same format.

2.3 – Abstract interface class

We introduced an abstract class AbstractGraph with pure virtual methods for all the core operations: *insert_edge, load_from_file, write_to_file, disconnect, remove_node, get_nodes*, and *get_out_neighbors*. Both Graph and MatrixGraph inherit from this, so any algorithm can work on either implementation without knowing which one it's using.

2.4 – Matrix-based data structure

The MatrixGraph class stores the graph as a 2D adjacency matrix. Each cell matrix[i][j] holds a vector of edge labels for all edges going from node i to node j, which handles multiple edges between the same pair of nodes. Node labels are tracked in a separate vector, and an unordered_map maps labels to their integer index for fast lookup.

2.5 – Delete edges and nodes

We implemented disconnect (a, b) which removes all edges from node a to node b, and remove_node(label) which deletes a node along with all its incident edges. After any deletion, we check whether any nodes have become isolated (no remaining edges) and remove those too.

2.6 – Rule of five

For the Graph class (which manages raw pointers), we manually implemented all five special member functions: destructor, copy constructor, copy assignment, move constructor, and move assignment. The copy operations do a deep copy of all nodes and edges, rebuilding the pointer connections. The move operations transfer ownership and leave the source in an empty but valid state. For MatrixGraph, all data is stored in STL containers so we can use the compiler-generated Defaults.

3.1 – Strongly connected components (Tarjan's algorithm)

We implemented Tarjan's algorithm in the Tarjan class (namespace tarjans_algorithm). The algorithm does a depth-first search and keeps track of discovery IDs and low-link values. A node is The root of a strongly connected component (SCC) when its ID equals its low-link value. Nodes are kept on a stack during the DFS, and an entire SCC is popped off the stack when its root is found. The algorithm takes an AbstractGraph reference, so it works with both the linked and matrix-based implementation. The program takes command-line flags --verbose and --silent to control output, and --linked or --matrix to choose the data structure. In verbose mode, all SCCs are printed; in silent mode only, the total count is shown.

3.2 – Diamond path query

The Diamond class (namespace diamond_finder) searches for pairs of nodes (ni, nj) that are connected by two parallel paths with specific edge label sequences. The query is read from a file With two lines, each line lists the edge labels for one of the two paths. The algorithm builds an internal adjacency list from the graph and then searches for matching path pairs using DFS. Like Tarjan, Diamond also takes an AbstractGraph reference, making it compatible with both storage implementations.

3.3 – Demonstration using GNU make

The Makefile was updated to support four separate executables, one for each problem: graph_pro22, graph_pro25, graph_scc, and graph_diamond, corresponding to the different main files. Common source files are compiled once and linked with each executable, avoiding issues with multiple main functions. Demos for problem 2.2, 2.5 3.1 and 3.2 are included.

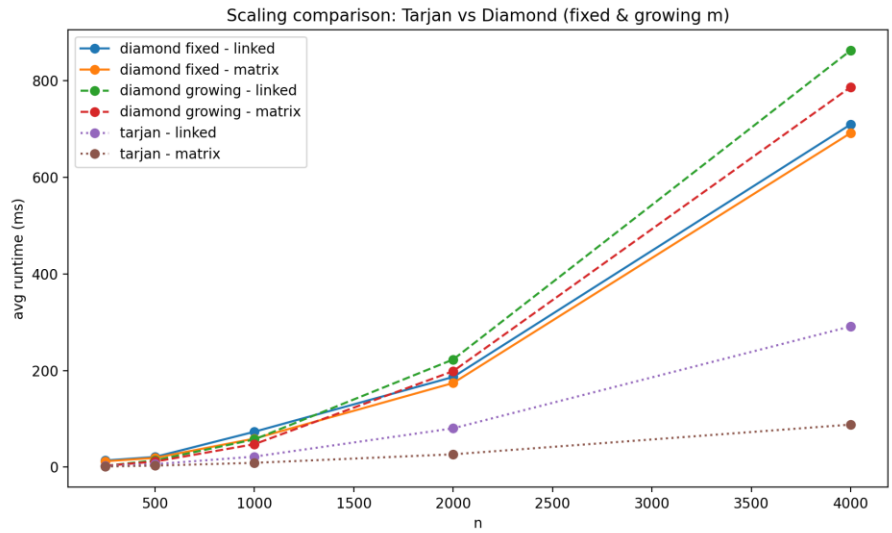
3.4 – Scaling and runtime measurement

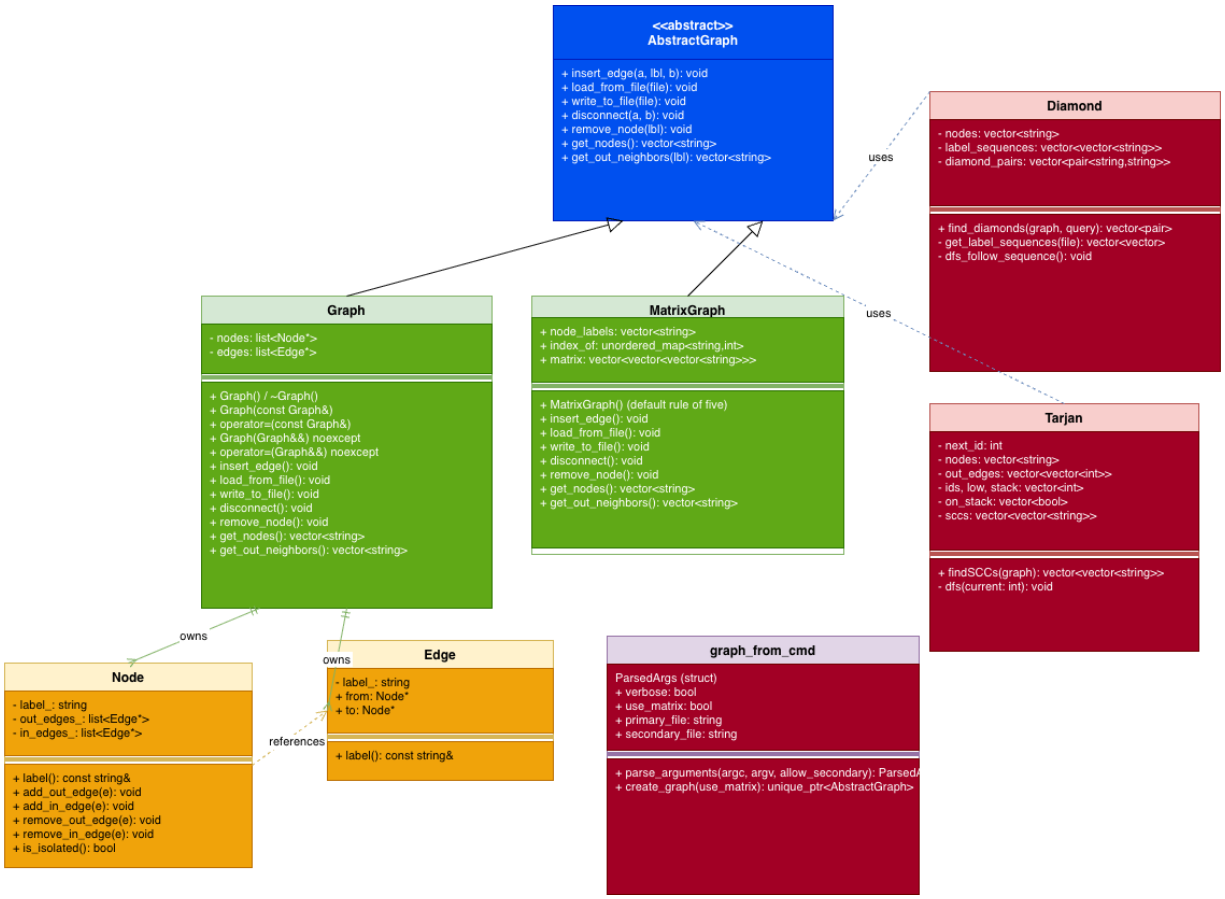
We generated graphs of increasing size and measured average runtime for both algorithms and graph representations. Each configuration was run multiple times.

Tarjan’s algorithm shows steady growth consistent with O(V + E) and performs slightly faster with the matrix representation.

The diamond path algorithm scales much worse, especially when the query size m grows with n. For fixed m, the growth is more moderate.

Overall, Tarjan scales efficiently, while the diamond algorithm is significantly more expensive. The matrix representation is consistently slightly faster than the linked representation.



3.5 – Entity-relationship diagram

The ER diagram below shows the full class architecture of the project. The design centers around The abstract AbstractGraph interface is implemented by two concrete classes. The algorithm classes depend only on this interface, not on any specific implementation.